BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 10

Roll no: 57 PRN No: 12410559 Name of Student: Avni Selote

Subject Name: Deep Learning

Problem Statement:

Design and implement a deep learning application for audio classification that incorporates Speech-to-Text (STT) and Text-to-Speech (TTS) using an appropriate deep learning architecture, and evaluate its performance using suitable metrics.

Algorithm:

- **Import Libraries:** Import the required libraries for audio processing, deep learning, evaluation, speech recognition, and text-to-speech.
- **Load Dataset:** Upload and extract the RAVDESS dataset and collect speech audio files with their corresponding emotion labels.
- **Select Samples:** Select a balanced subset of audio samples for the eight emotion classes to reduce training time.
- **Audio Preprocessing:** Resample each audio file to 22,050 Hz, standardize its duration to 3 seconds, and extract a 64-bin Mel spectrogram.
- **Prepare Dataset:** Encode the emotion labels, reshape the features for CNN input, and split the dataset into training and testing sets.
- **Build CNN Model:** Construct a CNN using convolutional layers, pooling, global average pooling, dropout, and a softmax output layer.
- **Train Model:** Train the CNN on the training dataset and validate it using the validation split.
- **Evaluate Model:** Calculate test accuracy, generate a classification report, display a confusion matrix, and plot training and validation accuracy.
- **Save Model:** Save the trained CNN model for future predictions.
- **Load Whisper:** Initialize the Whisper speech recognition model.
- **Speech-to-Text:** Upload a WAV audio file and transcribe its spoken content using Whisper.

- **Emotion Prediction:** Preprocess the uploaded audio using the same feature extraction procedure and predict its emotion using the trained CNN.
- **Text-to-Speech:** Convert the recognized text into speech using gTTS and play the generated audio.
- **Display Results:** Display the transcription, predicted emotion, prediction confidence, evaluation results, and generated speech

Objectives :

1. To classify speech audio into six emotions using a CNN.
2. To preprocess audio and extract Mel Spectrogram features.
3. To integrate Speech-to-Text (STT) and Text-to-Speech (TTS).
4. To evaluate performance using accuracy, precision, recall, and F1-score.
5. To improve classification performance through data augmentation and model tuning.

Code and Output:

```
import os, re, glob, random, warnings, time, subprocess, sys
from pathlib import Path
from collections import Counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import librosa
import soundfile as sf
from tqdm.auto import tqdm

from sklearn.model_selection import GroupShuffleSplit
from sklearn.metrics import (
    accuracy_score, precision_recall_fscore_support,
    classification_report, confusion_matrix
)

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
```

```
import torch
import torch.nn as nn
import random

def augment_mel(spec):
    """
    spec shape: [1, n_mels, time_frames]
    Apply augmentation only to training samples.
    """
    spec = spec.clone()

    # Small gain variation
    spec = spec * random.uniform(0.85, 1.15)

    # Add a small amount of noise
    spec = spec + torch.randn_like(spec) * 0.01

    # Frequency masking
    if random.random() < 0.5:
        max_freq_mask = min(8, spec.shape[1])
        width = random.randint(0, max_freq_mask - 1)
        if width > 0:
            start = random.randint(0, spec.shape[1] - width)
            spec[:, start:start + width, :] = 0

    # Time masking
    if random.random() < 0.5:
        max_time_mask = min(20, spec.shape[2])
        width = random.randint(0, max_time_mask - 1)
        if width > 0:
            start = random.randint(0, spec.shape[2] - width)
            spec[:, :, start:start + width] = 0

    return spec
```

```
import kagglehub

path = kagglehub.dataset_download("ejlok1/cremad")
print("Path to dataset files:", path)
DATA_ROOT = Path(path)
```

```
Downloading to /root/.cache/kagglehub/datasets/ejlok1/cremad/1.archive...
100%|██████████| 451M/451M [00:27<00:00, 17.4MB/s]Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/ejlok1/cremad/versions/1
```

+ Code + Text

```
audio_files = []
for pattern in ("**/*.wav", "**/*.WAV", "**/*.mp3", "**/*.MP3"):
    audio_files.extend(DATA_ROOT.glob(pattern))
audio_files = sorted(set(p for p in audio_files if p.is_file()))
print("Audio files found:", len(audio_files))
print("Example paths:")
for p in audio_files[:10]:
    print(p)
if not audio_files:
    raise FileNotFoundError(f"No audio files found below {DATA_ROOT}. Inspect the downloaded folder structure.")

EMOTION_MAP = {
    "ANG": "anger",
    "DIS": "disgust",
    "FEA": "fear",
    "HAP": "happy",
    "NEU": "neutral",
    "SAD": "sad",
}
rows = []
for p in audio_files:
    # CREMA-D basename: actor_sentence_emotion_intensity.ext
    parts = p.stem.upper().split("_")
    if len(parts) >= 4 and parts[2] in EMOTION_MAP:
```

[6] ✓ 0s

```
EMOTION_MAP = {
    "ANG": "anger",
    "DIS": "disgust",
    "FEA": "fear",
    "HAP": "happy",
    "NEU": "neutral",
    "SAD": "sad",
}
rows = []
for p in audio_files:
    # CREMA-D basename: actor_sentence_emotion_intensity.ext
    parts = p.stem.upper().split("_")
    if len(parts) >= 4 and parts[2] in EMOTION_MAP:
        try:
            actor_id = parts[0]
            rows.append({
                "path": str(p),
                "filename": p.name,
                "actor_id": actor_id,
                "sentence_code": parts[1],
                "emotion_code": parts[2],
                "emotion": EMOTION_MAP[parts[2]],
                "intensity": parts[3],
            })
        except Exception:
            pass

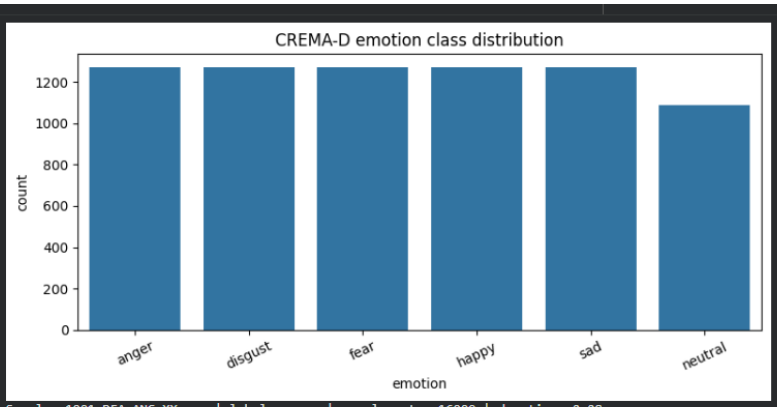
df = pd.DataFrame(rows).drop_duplicates(subset=["path"]).reset_index(drop=True)
print("Recognized CREMA-D files:", len(df))
display(df.head())
print("\nClass counts:")
display(df["emotion"].value_counts().reindex(list(EMOTION_MAP.values())).fillna(0).astype(int).to_frame("count"))
if len(df) < 100:
    print("WARNING: Few files were recognized. Check the filename pattern and folder contents.")
```

/root/.cache/kagglehub/datasets/ejlok1/cremad/versions/1/AudioWAV/1001_IEO_DIS_HI.wav
Recognized CREMA-D files: 7442

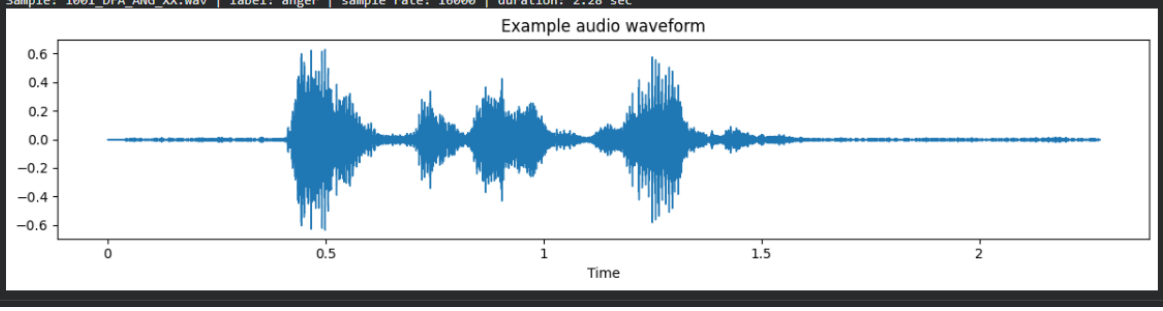
	path	filename	actor_id	sentence_code	emotion_code	emotion	intensity
0	/root/.cache/kagglehub/datasets/ejlok1/cremad/...	1001_DFA_ANG_XX.wav	1001	DFA	ANG	anger	XX
1	/root/.cache/kagglehub/datasets/ejlok1/cremad/...	1001_DFA_DIS_XX.wav	1001	DFA	DIS	disgust	XX
2	/root/.cache/kagglehub/datasets/ejlok1/cremad/...	1001_DFA_FEA_XX.wav	1001	DFA	FEA	fear	XX
3	/root/.cache/kagglehub/datasets/ejlok1/cremad/...	1001_DFA_HAP_XX.wav	1001	DFA	HAP	happy	XX
4	/root/.cache/kagglehub/datasets/ejlok1/cremad/...	1001_DFA_NEU_XX.wav	1001	DFA	NEU	neutral	XX

Class counts:

emotion	count
anger	1271
disgust	1271
fear	1271
happy	1271
neutral	1087
sad	1271



Sample: 1001_DFA_ANG_XX.wav | label: anger | sample rate: 16000 | duration: 2.28 sec



```python
# First reserve test actors, then split remaining actors into train and validation.
split1 = GroupShuffleSplit(n_splits=1, test_size=0.20, random_state=SEED)
trainval_idx, test_idx = next(split1.split(df, groups=df["actor_id"]))
trainval_df = df.iloc[trainval_idx].reset_index(drop=True)
test_df = df.iloc[test_idx].reset_index(drop=True)

split2 = GroupShuffleSplit(n_splits=1, test_size=0.20, random_state=SEED + 1)
train_idx, val_idx = next(split2.split(trainval_df, groups=trainval_df["actor_id"]))
train_df = trainval_df.iloc[train_idx].reset_index(drop=True)
val_df = trainval_df.iloc[val_idx].reset_index(drop=True)

print("Rows:", len(train_df), len(val_df), len(test_df))
print("Unique actors:", train_df.actor_id.nunique(), val_df.actor_id.nunique(), test_df.actor_id.nunique())
print("Actor overlap train/val:", set(train_df.actor_id) & set(val_df.actor_id))
print("Actor overlap train/test:", set(train_df.actor_id) & set(test_df.actor_id))
print("Actor overlap val/test:", set(val_df.actor_id) & set(test_df.actor_id))
for name, part in [("train", train_df), ("validation", val_df), ("test", test_df)]:
    print(name, part["emotion"].value_counts().to_dict())
```

```
Rows: 4672 1218 1552
Unique actors: 57 15 19
Actor overlap train/val: set()
Actor overlap train/test: set()
Actor overlap val/test: set()
train {'anger': 798, 'disgust': 798, 'fear': 798, 'happy': 798, 'sad': 798, 'neutral': 682}
validation {'anger': 208, 'disgust': 208, 'fear': 208, 'happy': 208, 'sad': 208, 'neutral': 178}
test {'anger': 265, 'disgust': 265, 'fear': 265, 'happy': 265, 'sad': 265, 'neutral': 227}
```

```python
SR = 16000
DURATION = 4.0
N_SAMPLES = int(SR * DURATION)
N_MELS = 64
N_FFT = 1024
HOP_LENGTH = 256
EMOTIONS = ["anger", "disgust", "fear", "happy", "neutral", "sad"]
LABEL_TO_ID = {label: i for i, label in enumerate(EMOTIONS)}
ID_TO_LABEL = {i: label for label, i in LABEL_TO_ID.items()}

def extract_logmel(file_path):
    audio, _ = librosa.load(file_path, sr=SR, mono=True)
    if len(audio) < N_SAMPLES:
        audio = np.pad(audio, (0, N_SAMPLES - len(audio)))
    else:
        audio = audio[:N_SAMPLES]
    mel = librosa.feature.melspectrogram(
        y=audio, sr=SR, n_fft=N_FFT, hop_length=HOP_LENGTH,
        n_mels=N_MELS, fmin=20, fmax=SR // 2, power=2.0
    )
    logmel = librosa.power_to_db(mel, ref=np.max)
    # Per-example normalization stabilizes input scale.
    logmel = (logmel - logmel.mean()) / (logmel.std() + 1e-6)
    return logmel.astype(np.float32)

# Precompute features once; this is CPU work and may take a few minutes.
feature_cache = {}
bad_files = []
for row in tqdm(df.itertuples(index=False), total=len(df), desc="Extracting log-Mel features"):
    try:
        feature_cache[row.path] = extract_logmel(row.path)
    except Exception as e:
        bad_files.append((row.path, str(e)))
print("Features cached:", len(feature_cache), "| failed files:", len(bad_files))
if bad_files:
    print("First errors:", bad_files[:3])

# Keep only files with successfully extracted features.
df = df[df["path"].isin(feature_cache)].reset_index(drop=True)
```

[11] ✓ 0s

```python
class CREMADataset(Dataset):
    def __init__(self, frame, cache):
        self.frame = frame.reset_index(drop=True)
        self.cache = cache
    def __len__(self):
        return len(self.frame)
    def __getitem__(self, idx):
        row = self.frame.iloc[idx]
        x = self.cache[row["path"]]
        x = torch.from_numpy(x).unsqueeze(0)  # [channel=1, mel, time]
        y = torch.tensor(LABEL_TO_ID[row["emotion"]], dtype=torch.long)
        return x, y

BATCH_SIZE = 32
NUM_WORKERS = 0  # portable across Windows, notebooks, and Colab
train_loader = DataLoader(CREMADataset(train_df, feature_cache), batch_size=BATCH_SIZE, shuffle=True, num_workers=NUM_WORKERS)
val_loader = DataLoader(CREMADataset(val_df, feature_cache), batch_size=BATCH_SIZE, shuffle=False, num_workers=NUM_WORKERS)
test_loader = DataLoader(CREMADataset(test_df, feature_cache), batch_size=BATCH_SIZE, shuffle=False, num_workers=NUM_WORKERS)

xb, yb = next(iter(train_loader))
print("Batch tensor shape:", xb.shape, "| labels:", yb.shape)
```

```
Batch tensor shape: torch.Size([32, 1, 64, 251]) | labels: torch.Size([32])
```

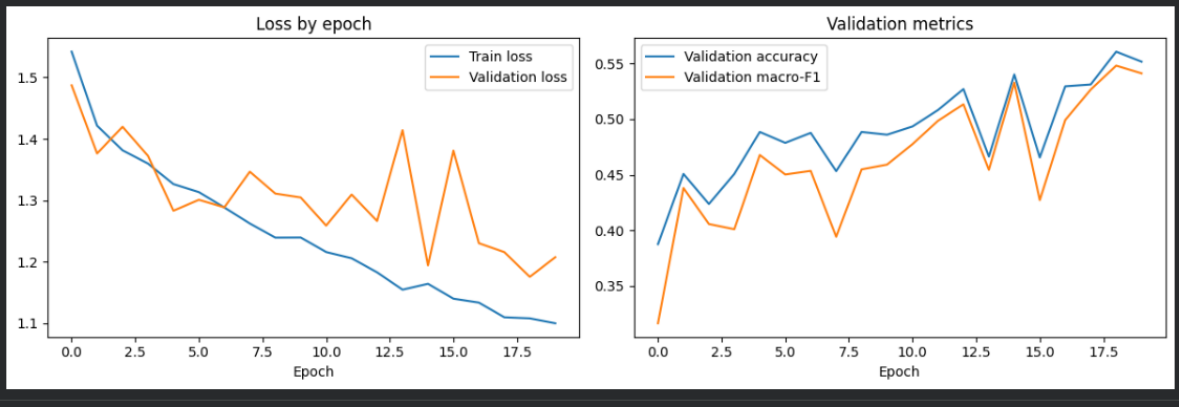
[12] ✓ 0s

```python
class EmotionCNN(nn.Module):
    def __init__(self, num_classes=6):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(),
            nn.AdaptiveAvgPool2d((4, 4))
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 4 * 4, 128),
            nn.ReLU(),
            nn.Dropout(0.35),
            nn.Linear(128, num_classes)
        )
    def forward(self, x):
        return self.classifier(self.features(x))

model = EmotionCNN(num_classes=len(EMOTIONS)).to(DEVICE)
print(model)
print("Trainable parameters:", sum(p.numel() for p in model.parameters() if p.requires_grad))
```

```
EmotionCNN(
  (features): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU()
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU()
    (11): AdaptiveAvgPool2d(output_size=(4, 4))
  )
```

```python
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(history["train_loss"], label="Train loss")
axes[0].plot(history["val_loss"], label="Validation loss")
axes[0].set_title("Loss by epoch"); axes[0].set_xlabel("Epoch"); axes[0].legend()
axes[1].plot(history["val_accuracy"], label="Validation accuracy")
axes[1].plot(history["val_macro_f1"], label="Validation macro-F1")
axes[1].set_title("Validation metrics"); axes[1].set_xlabel("Epoch"); axes[1].legend()
plt.tight_layout()
plt.show()
```

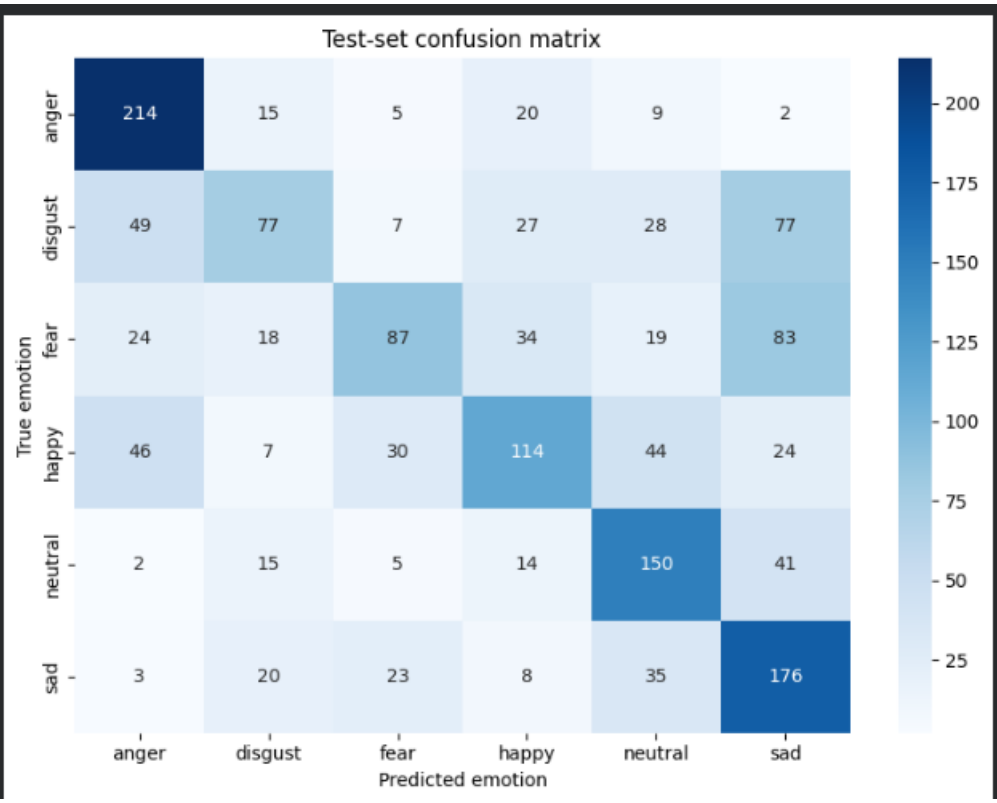



```
Test loss: 1.2302
Test accuracy: 0.5271
Macro precision: 0.5304
Macro recall: 0.5303
Macro F1-score: 0.5129

Classification report:
              precision    recall  f1-score   support

       anger       0.63      0.81      0.71       265
     disgust       0.51      0.29      0.37       265
        fear       0.55      0.33      0.41       265
       happy       0.53      0.43      0.47       265
     neutral       0.53      0.66      0.59       227
         sad       0.44      0.66      0.53       265

    accuracy                           0.53      1552
   macro avg       0.53      0.53      0.51      1552
weighted avg       0.53      0.53      0.51      1552
```



```
File: /root/.cache/kagglehub/datasets/ejlok1/cremad/versions/1/AudioWAV/1001_DFA_ANG_XX.wav
Predicted emotion: anger
Confidence: 78.47%
```

	emotion	probability
0	anger	0.784714
3	happy	0.126873
1	disgust	0.062681
4	neutral	0.016134
2	fear	0.007974
5	sad	0.001625

```
Saved model: cremad_emotion_cnn_final.pt
Saved predictions: cremad_test_predictions.csv
/usr/local/lib/python3.13/dist-packages/pandas/io/formats/format.py:1458: RuntimeWarning: overflow encountered in cast
  has_large_values = (abs_vals > 1e6).any()
```

	true_emotion	predicted_emotion	confidence
0	anger	anger	0.784668
1	disgust	happy	0.346924
2	fear	happy	0.449463
3	happy	happy	0.499756
4	neutral	neutral	0.618164